

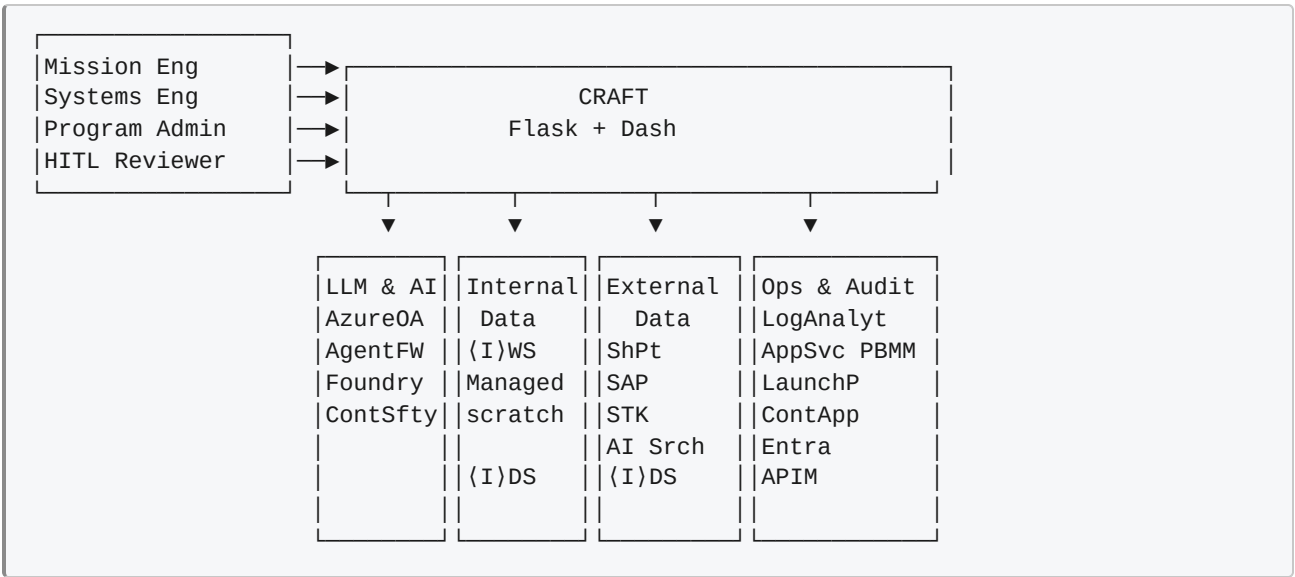
Legend

■ Actor · ◇ Gate · ▲ Cloud · ○ Terminal · ▒ Store · ⟨I⟩ Protocol · Blue=Auth · Green=Audit

1. What CRAFT Is

Proposed audited retrieval-augmented AI assistant for CSA mission engineering and finance. This document describes the target architecture — the seams, the components behind each seam, and the invariants that hold across the system.

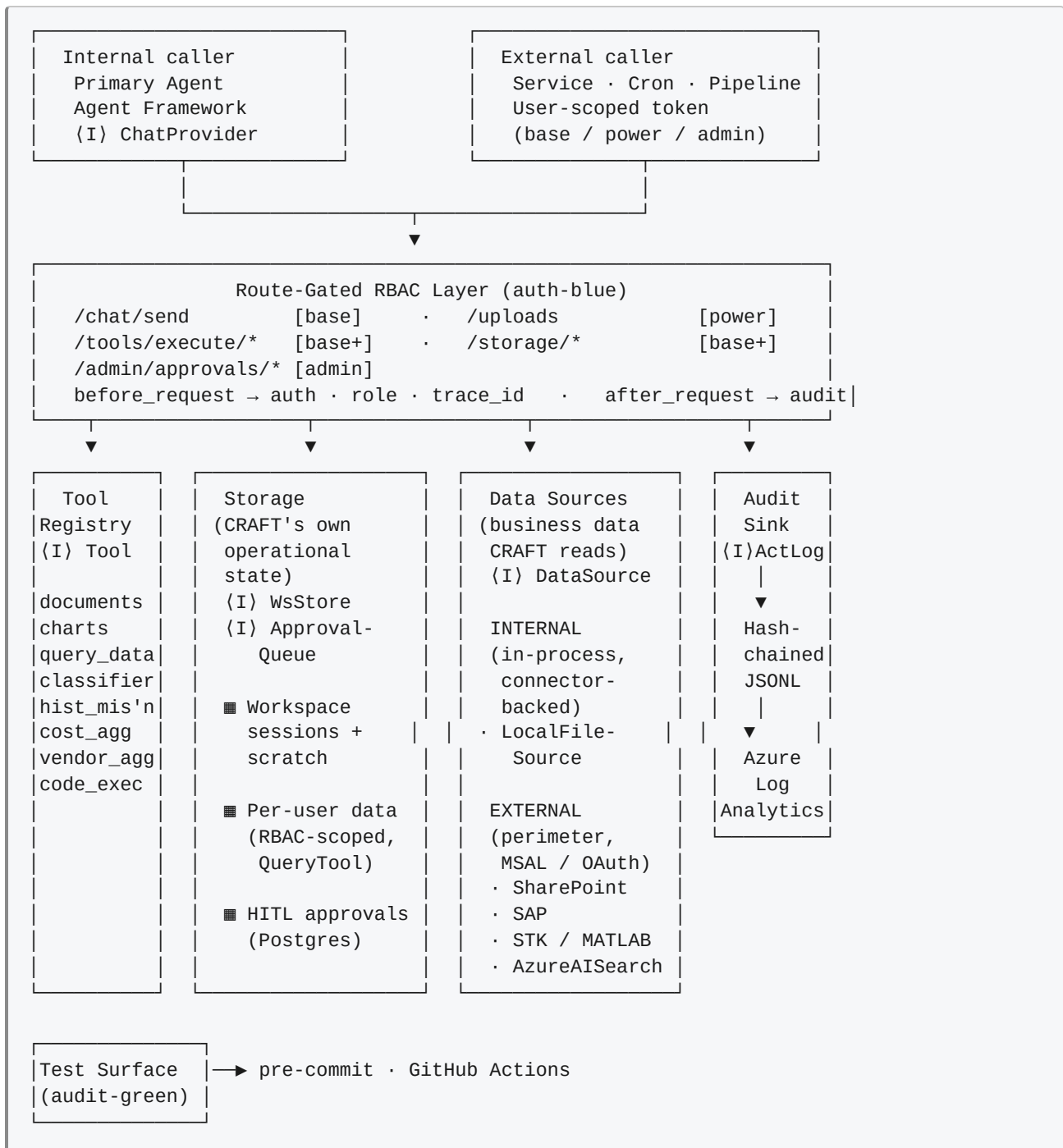
2a. L1 System Context



In: /chat/send [base] /admin/* [admin] . Out: thin italic. **Internal data** lives in the CRAFT process and the managed DB; **external data** reaches CRAFT through ⟨I⟩ DataSource adapters (MSAL/OAuth).

2b. L2 Container

The architecture reduces to three horizontal bands: **a route-gated RBAC layer, the surfaces it dispatches to** (tools, storage, data sources), and **an audit envelope** that wraps every call. Two caller shapes — the primary agent and any external token-bearing client — converge on the same gate; any exposed endpoint can be hit from anywhere, with anything, and returns a structured response. The design is agnostic to what we run it on, where it's called from, and how it's called — a terminal, a cron, a pipeline, a webhook target, or the agent itself all look the same to the gate.



Storage vs Data Sources. They are different concerns.

- **Storage** holds CRAFT's *own* operational state. Two protocols, three surfaces:
 - `{I} WorkspaceStore` — workspace sessions, scratch, and per-user operational data (RBAC-scoped via `QueryTool.set_user`). Backed by the managed DB.
 - `{I} ApprovalQueue` — the HITL approval queue. Backed by Postgres (durable across restarts, multi-reader for the agent-resume path and the reviewer UI).
- **Data Sources** are where business data *lives* — read-mostly, often outside CRAFT's trust boundary. One `{I} DataSource` contract covers both:
 - **INTERNAL:** `LocalFileSource` reads the mission corpus on disk.
 - **EXTERNAL:** SharePoint, SAP, STK/MATLAB, Azure AI Search — each a `{I} DataSource` class with MSAL / OAuth at the perimeter.

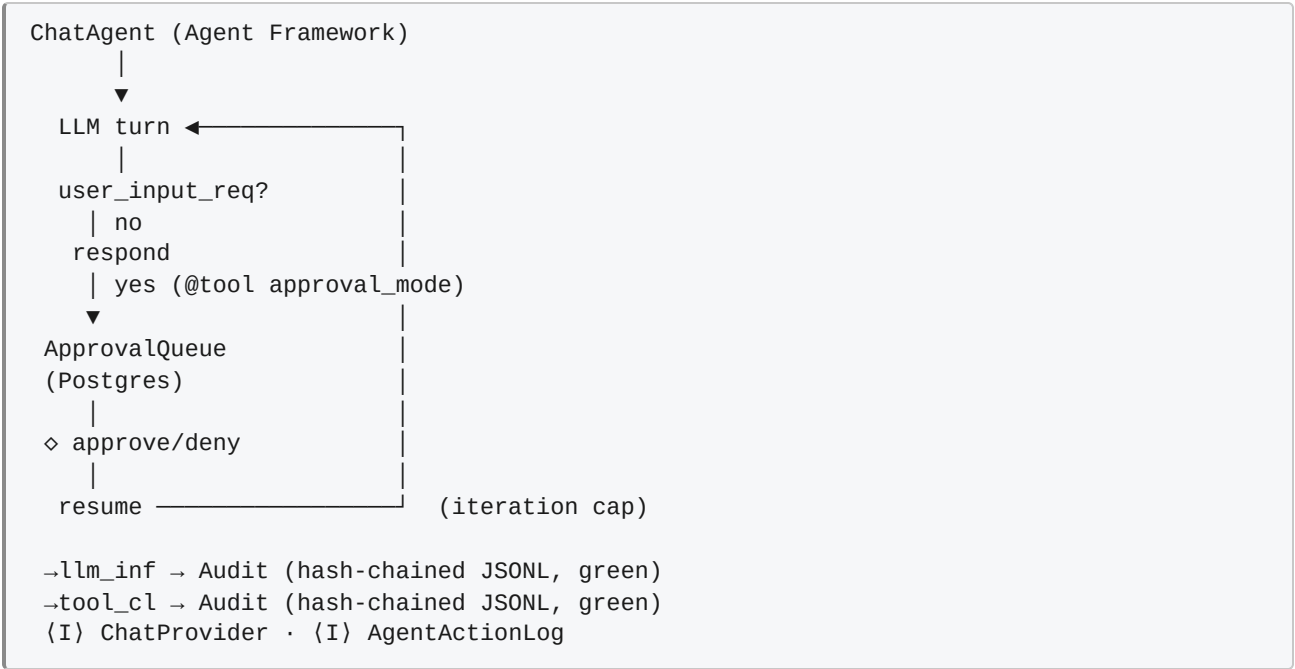
They're kept separate because they have different lifecycles and trust models: Storage is private state CRAFT owns end-to-end; Data Sources are read-mostly, cross the perimeter, and need per-request RBAC

scoping.

Tools are not just an LLM-callable surface. The same `/tools/execute/<name>` routes accept direct calls from external processes, and tools can carry a webhook callback — so a long-running simulation (STK / MATLAB on separate hardware, for example) can post its result back and resume the chat without blocking the agent.

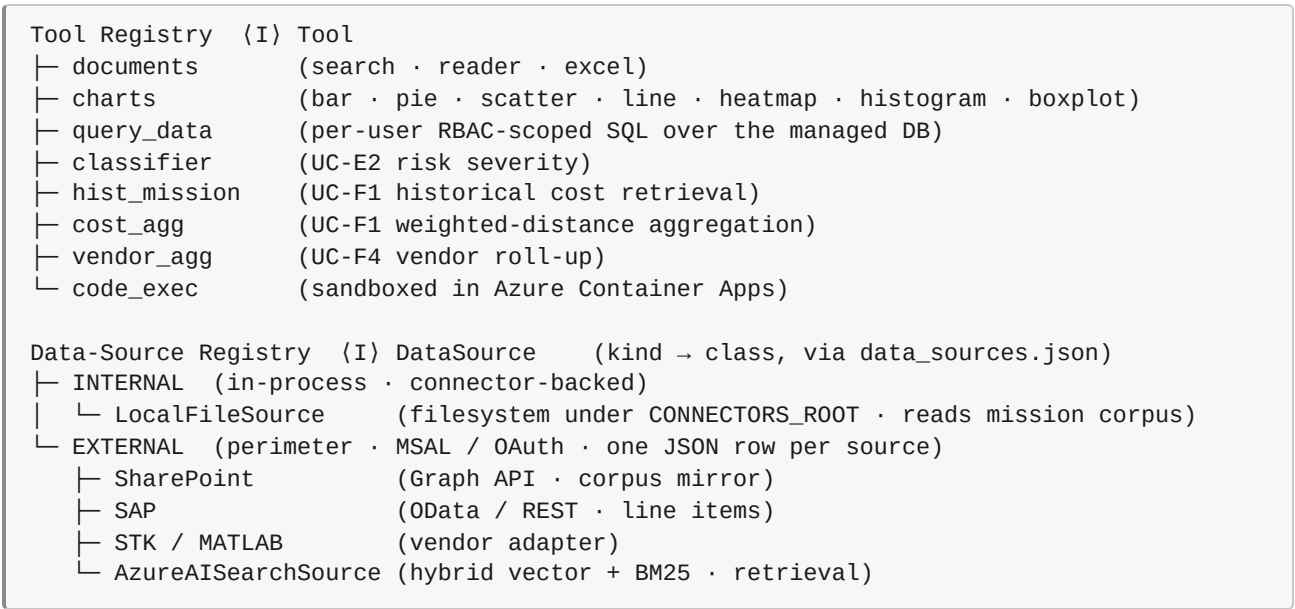
2c. L3 Internals

Panel A · Agent



Panel B · Tool & Data-Source Registries

The registries behind the route surface shown in 2b. Each {I} protocol has a kind registry; adding a new tool or connector is one class + one JSON row.



Panel C · Auth

```

Public inbound [/route][role]
  → FlaskRoute → ◇ before_request (hex, blue)
    auth · role · workspace · trace_id · flags
  → handler
  → after_request (audit, green)
  → response

permissions.json : /chat/send → base    /uploads → power    /admin/* → admin
RBAC              : base(1) < power(2) < admin(3)
flags             : experimental features gated per user (opt-in cohort)
(test) arch invariants (audit-green)

```

Experimental features and contributor on-ramp. Role flags gate experimental features so a cohort of users can trial them before general rollout — a straightforward way to run new tools and workflows against a small group (e.g., interested students on Mireille's team) and collect feedback. The same gating lets contributors from non-software-oriented teams land useful work safely: they build **Claude skills, workflows, and rules** (not low-level app code), their contributions target **feature branches only** — main is protected — and their output is exercised behind a flag until it's ready to graduate.

2d. UC State Models

```

UC-E2 · Risk ID + HITL (Engineering)
  Req → ◇ Auth → Retrieval [002-006] → Classifier [036-040] → ◇ HITL [HITL-001]
    Approve → RiskReg → Audit → •
    Reject  → Audit   → •

UC-F1 · Parametric Cost + HITL (Finance)
  Req → ◇ Auth → Retrieval (hist) [031] → CostAgg [032-034] → ◇ HITL [035]
    Approve → Commit → Audit → •
    Reject  → Audit  → •

UC-F4 · Vendor Cost (SAP)
  Req → ◇ Auth → SAPConn [053-055] → VendorExt → Agg → Audit → •

```

●=Terminal. ◇=Gate. Swim lanes: User → Agent → Services → Terminal.

3. Use Cases

UC-E2 · Risk ID + HITL (Engineering). Engineer uploads a CADRe part; retrieval pulls risk sections from the corpus; a classifier tool scores severity; HITL approves (framework `@tool(approval_mode="always_require")` → `ApprovalQueue` → resume route); a `DataSource` write lands the entry in the risk register. Tools: `DocumentSearchTool`, `ClassifierTool`. Requirements: UMR-002/006, UMR-036–040, HITL-001.

UC-F1 · Parametric Cost + HITL (Finance). Admin asks for a mission cost estimate; retrieval returns similar historical missions; an aggregator tool computes weighted distance; HITL approves; the commitment is written. Tools: `HistoricalMissionTool`, `CostAggregatorTool`. Requirements: UMR-031–035.

UC-F4 · Vendor Cost Roll-up (Finance). Analyst asks "how much did we spend with vendor X across missions?"; a SAP `DataSource` returns line items; a `VendorAggregatorTool` sums by vendor; a report is returned. Requirements: UMR-053–055.

UC · Bilingual Chat / RAG (All). User asks in English or French; retrieval + `DocumentSearchTool` return cited passages; the agent responds with citations and a confidence score. Requirements: UMR-001/002/003/007/010.

New mission flows (budget scenarios, anomaly triage, cross-mission compare) land as new `{I} Tool` + optional `{I} DataSource` behind the same lane.

4. Design Decisions

Component	Choice	Why
HTTP surface	Flask + Dash reviewer UI	Routes are the integration contract — any token-bearing caller (user, cron, agent) can hit CRAFT; Dash shares the auth and audit perimeter.
Agent runtime	Microsoft Agent Framework (<code>agent-framework</code> , Microsoft's enterprise successor to Semantic Kernel) · <code>ChatAgent</code> + <code>AzureOpenAIChatClient</code> · <code>@tool</code> with <code>approval_mode</code> · bounded iteration cap	Microsoft-blessed, Azure-native, long-term support. First-class HITL (function-level approval + workflow-level <code>RequestPort</code>), bounded iteration, OpenTelemetry built in. <code>{I} ChatProvider</code> keeps the swap seam honest.
LLM inference	Azure AI Foundry (model routing + rate limits + Content Safety) over Canadian PBMM region	Foundry collapses model, rate-limit and guardrail procurement into one line while PBMM residency stays. SDK is <code>agent_framework.openai.AzureOpenAIChatClient</code> .
Retrieval	Azure AI Search hybrid (vector + BM25) via <code>azure-search-documents</code> , exposed as <code>{I} DataSource</code>	Same <code>{I} DataSource</code> call site across all retrieval; cutover between backends is config, not code. Satisfies UMR-004/005/006.
Guardrails + sandbox	Azure Content Safety on the deployment + Azure Container Apps sandbox via <code>CodeExecTool</code>	Content filtering reads inline from the chat response; code-exec runs in an isolated container. UMR-020 / UMR-093. <code>GUARDRAIL_HOOK</code> sits in the tool wrapper so wiring is a config change.
Identity	Entra ID + MSAL behind <code>{I} AuthProvider</code>	Same token flow for humans and external agents; CSA-standard.
Audit sink	<code>HashChainedJsonLog</code> → Azure Log Analytics (immutable retention)	Append-only JSONL with SHA-256 chain for tamper detection; KQL at query time; same envelope end-to-end. UMR-027 / ASG-003.
HITL queue + memory	Postgres (approval queue + 3-tier memory) behind <code>{I} ApprovalQueue</code> via <code>psycpg[binary]</code>	Durable across restarts; UMR-025 TTL, UMR-014 memory. Dash reviewer UI consumes the queue.
Session + per-user data	Managed DB behind <code>{I} WorkspaceStore</code> (workspace + scratch surfaces)	Per-user scope via <code>QueryTool.set_user</code> ; one durable backing store for all operational state.
Edge	APIM + App Gateway (WAF, rate limit)	WAF + rate limit at perimeter; UMR-030.
Deployment	App Service PBMM + SSC LaunchPad HA	HA + landing zone come with LaunchPad; UMR-057.
Protocol contracts	<code>{I} Tool</code> , <code>{I} DataSource</code> , <code>{I} ChatProvider</code> , <code>{I} WorkspaceStore</code> , <code>{I} AgentActionLog</code> , <code>{I} ApprovalQueue</code> , <code>{I} AuthProvider</code>	New capability or backend = one class satisfying a protocol; agent unchanged. Pyright-strict verified.
Invariants	pytest + pyright (strict) + ruff + bandit + pip-audit + detect-secrets + doctest coverage + stateful-module coverage	Route × role matrix, protocol contracts, audit-envelope present, every public function has an executable example, every stateful module has a test file. Industry-standard, maintained.

5. Solving the Problems

Problem (requirement)	How the architecture solves it
Cited retrieval (UMR-002/005/006, ARR-002/003)	<code>DocumentSearchTool</code> + <code>TextAnalysisTool</code> call <code>AzureAISearchSource</code> through <code>{I} DataSource</code> ; retrieval response schema carries document / page / section provenance; agent surfaces citations to the user.
Grounding + confidence (UMR-007/008/010)	Grounding node sits between retrieve and respond; Foundry evaluation + RAGAS-style scoring emit confidence into <code>AGENT_ACTION</code> ; threshold check escalates below 0.6.
HITL approvals (UMR-021–026, HITL-001–007)	Agent Framework <code>@tool(approval_mode="always_require")</code> on risky tools → <code>user_input_requests</code> → <code>ApprovalQueue</code> (Postgres) → <code>/admin/approvals/<trace_id>/<decision></code> resume route, same auth as routes; decisions logged immutably with reasoning chain, confidence, sources; Dash reviewer UI.
RBAC + data scope (UMR-012/017/058, AAR-002/007)	Route-gated declarative <code>permissions.json</code> (role levels + public prefixes + internal prefixes + route rules); Entra ID issues tokens; the managed DB (behind <code>{I} workspaceStore</code>) is scoped by <code>QueryTool.set_user</code> ; classification tags enforced at the data store.
Immutable audit (UMR-015/027/045, ASG-003)	<code>HashChainedJsonLLog</code> writes append-only JSONL with SHA-256 hash chain, daily rotation, covering every tool call and LLM turn; forwarded to Azure Log Analytics for KQL and immutable retention.
Guardrails + sandbox (UMR-020/029/030/093, ASG-001/004/011)	Azure Content Safety on the deployment (<code>content_filter_results</code> read inline); <code>GUARDRAIL_HOOK</code> in the tool wrapper applies thresholds; <code>CodeExecTool</code> runs in Azure Container Apps; APIM rate limit at perimeter.
External data (UMR-051–054)	SharePoint, SAP, STK, MATLAB each a <code>{I} DataSource</code> class with MSAL / OAuth; one JSON row in <code>data_sources.json</code> ; <code>AzureAISearchSource</code> sits behind the same contract for retrieval.
Multi-agent + memory (UMR-011/014/018/095, AAR-001/004)	Primary <code>ChatAgent</code> + tool registry + session workspace; <code>agent_id</code> in <code>AGENT_ACTION</code> carries sub-task identity; orchestrator + specialist agents via framework multi-agent primitives + workflow-level <code>RequestPort</code> ; Postgres holds the three memory tiers; per-session agent factory.
Bilingual reports (UMR-001/046/048)	Agent accepts EN/FR free text; Azure Translator + Foundry NLG generate narrative sections; report generator attaches citations and runs through HITL.
Interoperability (UMR-055)	REST surface is the contract; routes sit behind APIM + App Gateway (WAF, rate limit); external agents trigger CRAFT with a token; GraphQL layer optional on top.
SLA + scale (UMR-056/057/059)	HA App Service + Azure Front Door; autoscale; SSC LaunchPad for the landing zone; health checks + load-test harness.

6. Requirements Traceability

Capability	Requirements	Design
Primary agent	UMR-001/011/013/016/018/019/094/095, AAR-001/003/006, ASG-005	Microsoft Agent Framework · ChatAgent + AzureOpenAIChatClient · {I} ChatProvider · bounded iteration cap
Retrieval + RAG	UMR-002/003/004/005/006/009/010/060/067, ARR-002–007	Azure AI Search hybrid · {I} DataSource registry (declarative JSON) · citation + grounding schema
Audit + traceability	UMR-015/027/045/061/062/093, AAR-005, ASG-003	<code>AgentAction</code> envelope · {I} AgentActionLog · hash-chained JSONL → Azure Log Analytics
HITL approvals	UMR-021–026/035/039/044/049, HITL-001–007, ASG-002	Agent Framework <code>@tool(approval_mode)</code> · {I} ApprovalQueue (Postgres) · Dash reviewer UI
RBAC + data scope	UMR-012/017/058/065/066, AAR-002/007	Declarative <code>permissions.json</code> (role levels + rules + prefixes) · Entra ID · managed DB behind {I} workspaceStore · <code>QueryTool.set_user</code>
Session + workspace	UMR-014/092/095	{I} workspaceStore → managed DB (workspace + scratch surfaces) · per-session agent factory

Capability	Requirements	Design
Guardrails + containment	UMR-020/029/030, ASG-001/004/011	<code>GUARDRAIL_H00K</code> + Azure Content Safety · APIM rate limit · Azure Container Apps sandbox
Connectors + external	UMR-051–055	<code>{I} DataSource</code> kind registry (JSON-driven) + MSAL · Flask REST · SAP / STK / MATLAB classes
Domain capabilities	UMR-007/008/031–050/075, ARR-001	New <code>{I} Tool</code> classes over existing registries (<code>ClassifierTool</code> , <code>HistoricalMissionTool</code> , <code>CostAggregatorTool</code> , <code>VendorAggregatorTool</code> , <code>CodeExecTool</code>)
Deployment + SLA	UMR-028/056/057/059/064/069/070/074/085, ASG-006	Azure App Service PBMM · APIM + WAF · SSC LaunchPad HA · stdlib + <code>python-dotenv</code> config loader

7. Dependencies & Blockages

Items the architecture depends on that need to be in place through IT, procurement, or domain partners.

Dependency	Owner	Satisfies
Microsoft Agent Framework (<code>agent-framework</code> SDK on PBMM)	IT (install access)	Primary agent runtime
Azure OpenAI (Canadian PBMM region)	IT (already in CSA tenant)	LLM inference
Azure AI Foundry (model routing + rate limits)	IT (procurement)	LLM routing, rate control
Azure AI Search (hybrid retrieval)	IT (deploy)	Cited retrieval
Azure Content Safety	IT (procurement)	Guardrails
Azure Container Apps	IT (deploy)	<code>CodeExecTool</code> sandbox
Entra ID (app registration + MSAL)	IT	Identity, token flow for humans + external agents
Azure Log Analytics (immutable retention policy)	IT	Immutable audit sink
APIM + App Gateway (WAF)	IT	Edge security, rate limiting
Postgres managed instance	IT	HITL approval queue + 3-tier memory
SharePoint access (Graph API, MSAL scope)	IT + finance	SharePoint <code>DataSource</code>
SAP access (OData endpoint, OAuth scope)	Finance + IT	SAP <code>DataSource</code> ; UC-F4
STK / MATLAB licenses + Python adapters	Engineering	STK/MATLAB <code>DataSource</code>
CSA risk taxonomy	Domain	<code>ClassifierTool</code> training / rules
Historical mission DB	Finance	<code>HistoricalMissionTool</code> ; UC-F1
SSC LaunchPad HA + landing zone	IT	Deployment

8. System-Health Tooling

Tool	Role
ruff	Lint (flake8 + isort + pyupgrade) · pre-commit + CI
pyright (strict)	Type checking · protocol conformance · pre-commit + CI
pytest + coverage	Route × role matrix · protocol contracts · audit-envelope invariants · pre-commit + CI
doctest	Every public pure function carries an executable example · <code>pytest --doctest-modules</code>

Tool	Role
architecture tests	AST-walkers in <code>app/tests/test_architecture.py</code> : doctest coverage, stateful-module test coverage, forbidden-import guard
bandit	Security scan · pre-commit
pip-audit	Dependency CVE audit · pre-commit + CI
detect-secrets	No credentials in git · pre-commit

Dependency surface: `agent-framework` (umbrella; or `agent-framework-core` + `agent-framework-openai` + `agent-framework-foundry`), `openai`, `dash`, `flask`, `flask-login`, `pandas`, `plotly`, `python-dotenv`, `requests`, `werkzeug`, `msal`, `pdfplumber`, `python-docx`, `azure-search-documents`, `psycpg[binary]`, `azure-monitor-opentelemetry`. Each row in §7 adds **one targeted dep** behind an existing protocol — not a tree.